

First Fit Memory Allocation in C

John Wensink

CSC507 Foundations of Operating Systems

Colorado State University – Global Campus

Dr. Joseph Issa

December, 8th, 2024

First Fit Memory Allocation in C

For this week's assignment, I implemented a basic solution of the first-fit memory allocation algorithm, to make fixed partitions of variable sizes that will store processes of variable sizes in main memory. The first-fit algorithm assigns each process to the first memory block that is large enough to accommodate it, aiming to allocate memory quickly and efficiently. To get started, I created a new project in Microsoft's Visual Studio 2022 (VS) in the C programming language, as learning the intricacies of C has been my goal since the beginning of this course. I added detailed comments to every line and every block of code to be sure that I understood the meaning and importance of every character in the program. With so much green comment text, I wanted to have the different steps of the program stand out a bit more, so I added the ***better comments*** extension into VS to allow for different colored comments for specific keywords. I ended up liking the `//TODO (comment)` keyword the best, as the yellow comment text stood out better with a dark-mode background. You can see the different sections of the code delineated in yellow descriptions of what the code block is about to do.

```
//TODO Step 1: Main function:
```

Before learning about C, I was familiar with using Python's interpreted language. C seems to operate at a much lower level, and I am finding that one needs to be much more explicit with with my code. Variable types need to be declared, syntax requires curly braces and semicolons, and memory needs to be manually allocated and freed as the language won't do it for you. For this increased complexity, C provides lightning-fast execution directly on the system's hardware.

The First-Fit Allocation Program

At the very top of the program, I added some preprocessing directives using the `#` modifier. I included ***stdio.h*** (standard I/O library) as the first line of the program to enable the ***printf*** and ***scanf*** commands. The preprocessing commands ***#define MAX_BLOCKS*** and ***#define MAX_PROCESSES*** allow for easy manipulation of the two main constants right up front so that these can be manipulated without having to change every instance of them within the program should one desire to do so. I followed the convention of having constants and macros written in all capital letters, with variable and function names written in camel case to indicate values that are likely to change in execution.

Step 1: The Main() Function

C differs from Python in that `main()`, being the entry point of the program, is mandatory for all for all C programs. In Python, `def main()` is optional, and the interpreter executes code sequentially starting at the top. In C, `main()` serves as the point which the operating system (OS) automatically calls to begin program execution. Here, it is defined with a return type of ***int*** to indicate the program's success or failure to the OS. Return 0; signifies successful execution, while any other value indicates an error. At this point we are ready to define our variables:

int blockSizes[MAX_BLOCKS]; is an array to store the sizes of memory blocks.

int processSizes[MAX_PROCESSES]; is an array to store the size of the processes.

int allocation[MAX_PROCESSES]; is an array to track which block each process is assigned to.

int m, n; are variables for the number of blocks (*m*), and number of processes (*n*.)

In C, multiple variables can be declared within the same statement as shown with *int m, n;*.

```
//TODO Step 1: Main function:
int main() {
    int blockSizes[MAX_BLOCKS]; //Array to store sizes of memory blocks.
    int processSizes[MAX_PROCESSES]; //Array to store the size of processes.
    int allocation[MAX_PROCESSES]; //Array to track which block each process is assigned to.
    int m, n; //Number of blocks (m) and processes (n). Multiple variables can be declared in the same statement in C.
```

Step 2: Initialize the Allocation Array

To initialize the allocation array, a *for* loop is used. Every *for* loop in C has four main components: initialization, condition, increment, and the loop body. The loop iterates from 0 to the length of the *MAX_PROCESSES* array, incrementing *i* by 1 every time the loop executes. The loop body sets every element in the allocation array to -1, to show that a process is not allocated. This step ensures that the array is in a known state before proceeding with the memory allocation logic.

```
//TODO Step 2: Initialize the allocation Array:
// A for loop in C has four main parts for(initialization; condition; increment){loop body}.
//Initialize the allocation array to -1 for each element up to the length of MAX_PROCESSES.
for (int i = 0; i < MAX_PROCESSES; i++) {
    allocation[i] = -1; // -1 here means that a process is not allocated.
}
```

Step 3: Input Memory Block Sizes

In this step, the program solicits user input for the number of memory blocks available for allocation. The *printf* function is used to display the maximum number of blocks allowed, corresponding to *MAX_BLOCKS*. The %d format specifier is a neat way to display the value of *MAX_BLOCKS* dynamically. The user input is then captured by *scanf* (or *scanf_s*, see the

‘problems’ section below), and the input is stored in the variable *m*, which represents the number of memory blocks. A validation check here ensures the user input is within a valid range, specifically, greater than zero and less than or equal to *MAX_BLOCKS*. If the input does not meet these criteria, the program prints an error message, and the program terminates and returns a value of 1. For future updates, I would like to implement a more graceful error handling system, a la Python’s *try* and *except*. This safeguard prevents the program from executing without valid input, ensuring reliable and predictable behavior in subsequent steps. When the number of memory blocks has been selected, the next block of code goes on to solicit user input for the sizes of each memory block. Another *printf* statement is used to display the message instructing the user to provide the sizes of the *m* memory blocks, and the *%d* modifier dynamically displays the value of *m*. Another *for* loop is called here to iterate *m* times corresponding to the number of memory blocks. Inside the loop, the program uses another *printf* statement to prompt the user to enter the size of a block element in the array. This time, however, we use *%d* to replace the element index with *i + 1* to make use of the easy reading 1-based block number for additional clarity as opposed to the 0-based block index. Again, *scanf* captures the user input and stores it in *blockSizes* at index *i*. This process ensures that each memory block’s size is recorded, setting up the program for the subsequent memory allocation logic.

```
//TODO Step 3: Input memory block sizes:
printf("Enter the number of memory blocks (max %d): ", MAX_BLOCKS); // %d is a format specifier for an integer value.
scanf_s("%d", &m); //scanf_s() is part of stdio.h and used to scan input provided to the user and store it in the specified variable m.
//We have to use scanf_s instead of plain scanf because visual studio considers scanf unsafe and gives errors at compilation time.
//Validation check to ensure the number of memory blocks is within the acceptable range.
if (m <= 0 || m > MAX_BLOCKS) { //If m is <= 0 || (logical OR) m is > MAX_BLOCKS (either condition will execute the if block).
    printf("Invalid number of memory blocks!\n"); //Error message will print to inform the user about the mistake.
    return 1; // Exit the program.
}

printf("Enter the sizes of the %d memory blocks:\n", m); //Message to ask the user to enter the size of the memory blocks and start a new line.
for (int i = 0; i < m; i++) { //For loop that iterates m times.
    printf("Block %d: ", i + 1); //Prompts user to enter the size of a specific block. The %d is replaced by i + 1 to show a 1-based block number.
    scanf_s("%d", &blockSizes[i]); //Reads the size of the current block user input and store the value in the ith element in the blockSizes array.
}
```

Step 4: Input Process Sizes

This step is very similar to Step 3, but focuses on gathering information about the processes instead of memory blocks. The program prompts the user to input the number of processes, with the max allowable number displayed using `%d`, represented by the constant `MAX_PROCESSES`. The user input is captured using `scanf` and stored in the variable `n`. Another validation check ensures that the input is greater than 0 and less than or equal to `MAX_PROCESSES`. Notice the `||` operator represents the logical **OR** operation in C. If the input fails to meet both of these criteria, the program displays an error and terminates with `return 1`. Next, the program prompts the user to enter the sizes of each of the `n` processes. Another **for** loop is used to iterate `n` times, and for each iteration, the user is prompted to input the size of a specific process using a 1-based index again. The input is captured using `scanf` and stored in the `processSizes` array at index `i`. This step prepares the program to allocate memory blocks to process, ensuring all process sizes are recorded in a valid, organized, and sequential manner.

```
//TODO Step 4: Input process sizes:
printf("\nEnter the number of processes (max %d): ", MAX_PROCESSES); //Prompt the user to input the number of processes starting on a new line.
scanf_s("%d", &n); //Read an integer input from the user and store it in the variable n.
if (n <= 0 || n > MAX_PROCESSES) { //If n <= 0 || (logical OR) n > MAX_PROCESSES (either condition will execute the if block).
    printf("Invalid number of processes!\n"); //Error message will inform the user about the mistake.
    return 1; // Exit the program.
}
printf("Enter the sizes of the %d processes:\n", n); //Prompt the user to input the size of the processes and start a new line.
for (int i = 0; i < n; i++) { //For loop that iterates n times.
    printf("Process %d: ", i + 1); //Prompts the user to enter the size of a specific process. The %d is replaced by i + 1 to show a 1-based process number.
    scanf_s("%d", &processSizes[i]); //Reads the size of the current process user input and store the value in the ith element in the processSizes array.
}
```

Step 5: First-Fit Allocation Logic

Here's where it gets fun. The first-fit allocation logic makes use of a nested **for** loop, with a conditional inner loop. The outer loop iterates for each process in the `processSizes` array, while the inner loop iterates through the `blockSizes` array to find the first block large enough to store

the current process. For each combination of process and block, the program checks if the current block's size (*blockSizes[j]*) is large enough to accommodate the current process's size (*processSizes[i]*.) If the condition is satisfied, the block is assigned to the process, which is recorded by setting *allocation[i]* to *j*. Additionally, the size of the allocated block is reduced by the size of the process (*blockSizes[j] -= processSizes[i]*), reflecting the available memory that is left over in that block. This part is important in the management of external fragmentation, as it tracks and updates the remaining sizes of memory blocks after allocation. A future version of this program might incorporate a compaction mechanism to reorganize memory by consolidating smaller fragments into larger contiguous blocks. Although compaction is computationally expensive, it could significantly improve memory efficiency in environments where fragmentation is a concern. If implemented alongside a system that allows for dynamic resizing of memory blocks during or after allocation, this program could evolve from a simple variable-partition allocation system to a full dynamic memory allocation system capable of adapting to changing process requirements and minimizing wasted space.

After assigning the block, the *break* statement exits the inner loop and moves on to the next iteration of the outer loop. The algorithm stops at the first block that fits each process, which makes it efficient in terms of time complexity but can lead to fragmentation. In contrast, the best-fit algorithm takes a different approach by searching all available blocks and selecting the smallest block that is large enough to accommodate the process. This approach reduces fragmentation by leaving larger blocks for processes requiring more memory (GeeksforGeeks, 2022.) While best-fit has the advantage of being space efficient in theory, in practice, it leaves leftovers that so small that they are unlikely to be big enough to fit a process, and compaction

must be run more frequently to make use of this external fragmentation space. Still, in environments where memory is a critical finite resource, and minimum fragmentation is essential to ensure optimal performance, best-fit might be a smart choice. Some examples of this might include cloud platforms like AWS or Azure, where infrastructure is shared among multiple users (Al-Yatama, Ahmad, & Al-Dabbous, 2017.) In such environments, best-fit ensures that smaller memory blocks are allocated to smaller VMs or containers, preserving larger blocks for memory-intensive workloads like databases or machine learning applications. By optimizing memory usage, cloud providers can lower their operational costs, improve scalability, and provide better performance for a wider range of customer needs.

Step 6: Print Allocation Results.

The final step prints the results of the memory allocation using a formatted table. The header is first printed to label the columns and uses `\t` escape character to create horizontal tabs for alignment, ensuring that the output is organized and easy to read. A *for* loop then iterates through each process, displaying its number and size along with the block where it was allocated (if any.) The process number and size are printed using *printf* with the `%d` specifier replaced by the appropriate values. If the allocation value is not -1, it means the process was successfully allocated to a block. Finally the program terminates with a *return 0*, signaling that the main function has completed without any errors.

```
//TODO Step 6: Print allocation results
printf("\nProcess No.\tProcess Size\tBlock No.\n"); //Print a table header to display the process allocation results \t makes a horizontal tab for alignment.
for (int i = 0; i < n; i++) { //loop through each process to display its size and the block it was allocated to.
    printf("%d\t%d\t", i + 1, processSizes[i]); // Print process number and size with tab spacing for alignment. Note the 2 instances of %d placeholders.
    if (allocation[i] != -1) { //If the current process i was assigned to a block (remember we initialized allocation array to be -1 to represent empty index).
        printf("%d\n", allocation[i] + 1); //Print the 1-based index of the block allocated to the current process and start a new line.
    }
    else { //If the current process has not been allocated to any block.
        printf("Not Allocated\n"); //Print that it has not been allocated and start a new line.
    }
}

return 0; //Program ran successfully.
```


Problems

VS did not like the *scanf* command when it came time to build the solution because it does not provide built-in safeguards to prevent potential buffer overflows. This caused me some frustration. The easy workaround is to replace all instances of *scanf* with *scanf_s*, which enforces additional safeguards. The problem with this is that *scanf_s* is strictly a feature for VS, and if someone wanted to run this program outside of VS, the program would fail to compile because other compilers don't recognize *scanf_s* (GeeksforGeeks, 2023.) The more challenging workaround, I found, was disabling warnings using *#define _CRT_SECURE_NO_WARNINGS* at the top of the program to accommodate the program for VS as well as other compilers. Within VS, there is also an option to add this to preprocessor definitions, but errors may arise if the definition and the preprocessing instruction are both set. To make this program portable, I chose to add the line at the top of the program and clear the definition setting to allow other systems to compile the code.

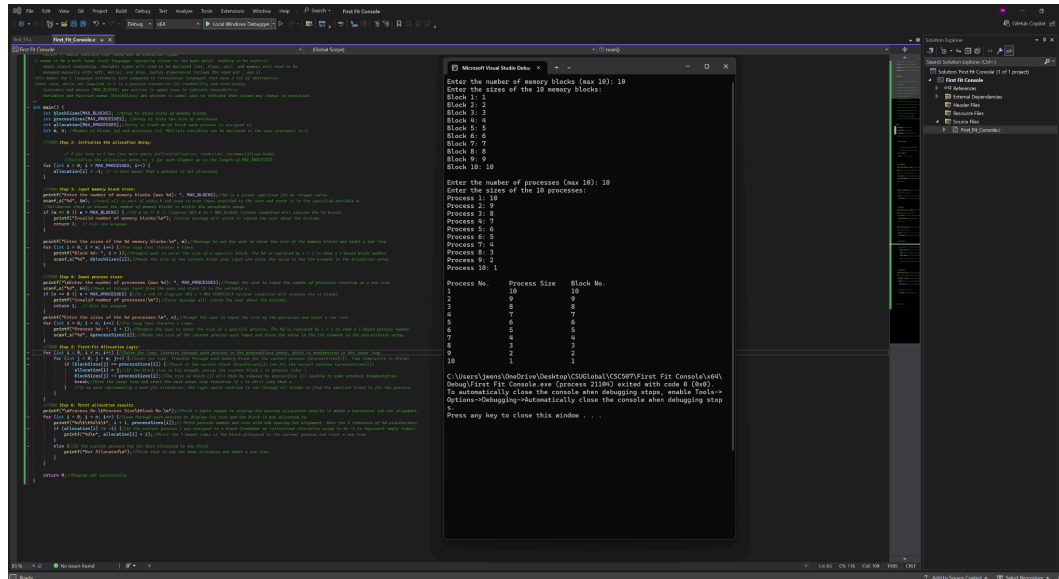
Execution

After fixing these errors, the program was able to execute and run without issue. The first test went perfectly, with block sizes starting from 1 at block 1 to 10 at block 10, and process sizes starting at 10 for process 1 and getting smaller to 1 at process 10. This forced the program

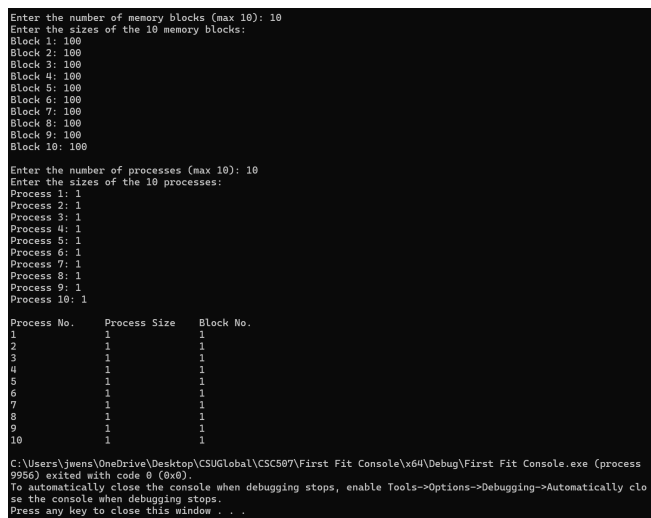
First Fit Memory Allocation in C

10

to run through the loop in the worst-case time complexity scenario.



A subsequent test with all block sizes being 10 and all process sizes being 1 revealed that the program was assigning multiple processes to a block.



To fix this, I added functionality to add a *blockAllocated* variable that is initialized so that each element is set to 0, indicating that the block is not yet allocated. During the allocation phase

First Fit Memory Allocation in C

11

when the inner loop iterates to find a block that is suitable, another conditional statement ensures that the block is not already allocated ***blockAllocated[j] == 0***. If the block gets allocated in the process, the corresponding variable is updated to 1. This prevents any other processes from being assigned to that block in subsequent iterations.

```
Microsoft Visual Studio Debug Console
Enter the number of memory blocks (max 10): 10
Enter the sizes of the 10 memory blocks:
Block 1: 10
Block 2: 10
Block 3: 10
Block 4: 10
Block 5: 10
Block 6: 10
Block 7: 10
Block 8: 10
Block 9: 10
Block 10: 10

Enter the number of processes (max 10): 10
Enter the sizes of the 10 processes:
Process 1: 1
Process 2: 1
Process 3: 1
Process 4: 1
Process 5: 1
Process 6: 1
Process 7: 1
Process 8: 1
Process 9: 1
Process 10: 1

Process No.    Process Size    Block No.
1             1              1
2             1              2
3             1              3
4             1              4
5             1              5
6             1              6
7             1              7
8             1              8
9             1              9
10            1             10

C:\Users\jwens\OneDrive\Desktop\CSUGlobal\CSC507\First Fit V4\x64\Debug\First Fit V4.exe (process 2624) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .
```

Conclusion

This project gave me a hands-on exploration of the first-fit memory allocation algorithm and the intricacies of implementing it in the C programming language. The algorithm's complexity and efficiency in allocation memory quickly made it an ideal choice for an educational assignment. Along the way, I encountered a few challenges, but nothing too terrible. I used an LLM Grimoire (2024) to help explain some concepts, but I tried not to lean on it too much as this was a relatively simple problem to solve. I

First Fit Memory Allocation in C

12

mostly needed help with C syntax and output formatting. The compatibility issue with using *scanf* means that it might not run outside of Microsoft VS. To that end, I also included an additional .c file with *scanf* and a preprocessing security exception at the top. While the first-fit algorithm offers speed and simplicity, it does come with the drawback of fragmentation, highlighting the tradeoffs between memory allocation strategies. Looking ahead, this project could be expanded to include dynamic memory resizing and compaction, as well as more robust error handling. Overall, this project demonstrated the practical challenges and opportunities of manual memory management, setting the stage for next week's curriculum around virtual memory.

References

Al-Yatama, A., Ahmad, I., & Al-Dabbous, N. (2017). Memory allocation algorithm for cloud services. *The Journal of Supercomputing*, 73(12), 5006–5033. Retrieved December 8, 2024, from <https://doi.org/10.1007/s11227-017-2069-8>

GeeksforGeeks. (2022, May 6). Best-fit allocation in an operating system. GeeksforGeeks. Retrieved December 8, 2024, from <https://www.geeksforgeeks.org/best-fit-allocation-in-operating-system/>

GeeksforGeeks. (2023, October 6). Inbuilt library functions for user Input | sscanf, scanf_s, fscanf_s. GeeksforGeeks. Retrieved December 8, 2024, from https://www.geeksforgeeks.org/sscanf-fscanf-sscanf-scanf_s-fscanf_s-sscanf_s/

Grimoire. (2024). OpenAI. Conversation about the C programming language. Retrieved December 8, 2024, from <https://chat.openai.com>